AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

WEEK 12 · LAB

Lab 8: First CUDA Kernels

Get on a GPU, write kernels that are correct, and find out how much of your runtime is the PCIe bus rather than the arithmetic.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

DURATION
One 2-hour block

PREREQUISITE
Unit 3, Exercise 17

SUBMIT
lab08-<ROLLNO>.pdf

MARKS
12

OFFERING
Odd Semester 2026-27

Getting a GPU

```
ssh <user>@asai-11-admin
export PATH=/usr/local/cuda/bin:$PATH
srun --gres=gpu:1 --cpus-per-task=8 --mem=16G --time=00:30:00 --pty bash
nvidia-smi
```

ON ASAICOMPUTE · COMPILE ON THE HEAD NODE, RUN THROUGH THE SCHEDULER

`nvcc` on the login node is fine. Running a kernel there is not. Everything in this lab that executes goes through `srun` or `sbatch`.

Build with `-arch=sm_80`: it runs on both the A100 nodes and the RTX 6000 Ada cards, so you do not have to care which node you were given.

Part 1: Correctness first (4 marks)

1.1 Write and run a vector add. Verify against a CPU reference.

1.2 Now **remove** the bounds guard `if (i < n)` and run with n = 1000 and 256 threads per block. Does it crash? Does it report an error? Check the result near the end of the array, and check an unrelated allocation.

1.3 Add the error-checking macro and wrap every call:

```
kernel<<<b,t>>>(...);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

Reintroduce the bug from 1.2. Is it reported now? At which call?

1.4 Deliberately request 2048 threads per block. What error do you get, and from which of the two checks?

PITFALL · THE LESSON OF PART 1

Two of the three beginner CUDA errors are **silent**. An out-of-bounds write corrupts memory with no message, and a launch failure surfaces at a later, unrelated call. Error checking is not optional hygiene; it is the only way the failures become visible at all.

Part 2: The bus (4 marks)

2.1 Time these three separately for a vector add over 100 million floats: host-to-device copy, kernel, device-to-host copy. Report all three and their percentages of the total.

2.2 Compute the effective bandwidth of each transfer in GB/s. Compare with the PCIe generation on your node (`nvidia-smi -q | grep -i pcie`).

2.3 Compare the total GPU time with a good OpenMP version on the same node's CPUs. Which wins for a **single** vector add? Explain using the numbers from 2.1.

2.4 Now run the kernel 100 times on data that stays resident, copying only once. Recompute the comparison. What does this tell you about when a GPU is the right tool?

> **DEFINITION · THE RULE THIS PART ESTABLISHES**
>
> PCIe delivers tens of GB/s; device memory delivers thousands. A GPU pays off only when the transfer cost is amortised over a lot of on-device work.

Part 3: Launch configuration (2 marks)

3.1 Time your vector add with 32, 64, 128, 256, 512 and 1024 threads per block. Plot. Is there a best value? Is the curve flat over a wide range?

3.2 Ask the compiler what resources you used:

```
nvcc -O2 -arch=sm_80 --ptxas-options=-v vecadd.cu -o vecadd
```

Report registers per thread and shared memory per block. Do these explain the shape of your plot?

Part 4: Divergence (2 marks)

4.1 Write a kernel whose body branches on `threadIdx.x % 2`, with both branches doing substantial and equal work. Time it.

4.2 Change the condition to `threadIdx.x / 32` so whole warps agree, keeping the same total work. Time it again.

4.3 Report the ratio and explain it in terms of how a warp executes a branch.

What to submit

`lab08-<ROLLNO>.pdf`: your answers, the transfer breakdown table from 2.1, the resident-data comparison from 2.4, the block-size plot, the divergence measurement, and all source as text.

MARKS	FOR
4	Part 1: all three failure modes reproduced and correctly attributed
4	Part 2: transfer breakdown measured and the amortisation argument made
2	Part 3: block-size sweep with the occupancy figures
2	Part 4: divergence measured and explained